

StreamDrop

Architecture & System Design

Shivam Gautam · 2025

Current System Rust + Axum · SvelteKit · PostgreSQL · HLS	Scale Target Horizontally scalable · CDN-delivered · Cost-efficient	Core Constraint Time-to-stream over video quality
--	--	---

1. Requirements

1.1 Functional Requirements

- Users upload video files anonymously — no account or login required.
- Upload size limited to 1GB. Common formats: MP4, MKV, AVI, MOV, WebM.
- System generates a unique shareable link per video immediately on upload.
- Users stream the video by opening the shareable link in any browser.
- Time-to-stream is minimised over video quality.

1.2 Non-Functional Requirements

- Availability: watch page loads and serves video even during active transcoding.
- Durability: uploaded videos must survive process restarts.
- Scalability: architecture supports horizontal scaling without redesign.
- Maintainability: storage and data access layers abstracted for easy swapping.
- Security: file type validated before processing; path traversal prevented.

2. Core Entities

The system has one primary entity: the Video. Everything orbits this single record.

Field	Type	Description
id	UUID	Internal identifier. Used for polling by uploader.
slug	TEXT	8-char random public ID in shareable URL. e.g. xK92mPqR
status	TEXT ENUM	Pending → Processing → Ready → Completed Failed
original_path	TEXT	Path or S3 key of raw uploaded file.
hls_path	TEXT	Path or S3 key of HLS manifest. Null until Ready.
size_bytes	BIGINT	File size in bytes.
mime_type	TEXT	MIME type. e.g. video/mp4
created_at	TIMESTAMPTZ	Upload timestamp.

Video Status Lifecycle

```
Pending      → Upload received. Job queued. Share link already valid.
Processing    → Worker running. ffmpeg transcoding.
Ready         → First HLS segments available. Viewer can press play NOW.
                (transcoding still in progress – hls.js buffers new segments)
Completed     → All segments written. Full video available.
Failed        → Transcoding failed.

# KEY: Ready and Completed are separate.
# Viewer starts watching in ~15s regardless of file size.
```

3. API Interface

Four endpoints. All JSON. Errors return { "error": "message" }.

POST /api/videos — Upload Video

Field	Details
Request	multipart/form-data, field: file
Constraints	MIME type must start with video/. Max 1GB.
Response 200	{ video_id, slug, share_url }
Response 400	{ "error": "Only video files are allowed" }
Response 413	{ "error": "File too large, max 1GB" }

Share URL returned immediately

Transcoding is spawned as a background Tokio task. Caller gets the share link before transcoding starts.

GET /v/:slug — Get Video Status

Field	Details
Request	GET /v/xK92mPqR
Response (processing)	{ status: "Processing", manifest_url: null }
Response (ready)	{ status: "Ready", manifest_url: "https://.../index.m3u8" }
Response 404	{ "error": "Video not found" }

Frontend polls every 2 seconds. When status is Ready and manifest_url is present, hls.js starts playback automatically.

GET /api/videos/:id/status — Status by UUID

Field	Details
Use case	Uploader tracks their own video by UUID.
Response	Same shape as GET /v/:slug

GET /videos/:slug/*file — Serve HLS Files (MVP only)

Field	Details
Security	Rejects paths containing .. or starting with /
Production	Removed in production — hls.js fetches from CloudFront directly.

4. Data Flow

4.1 Upload Flow

```
Browser                Backend                PostgreSQL                ffmpeg
|— POST /api/videos -> |— stream to disk    |                |
|                        |— INSERT Pending ->|                |
|<— { slug, share_url }|                |                |
|                        |— tokio::spawn —>UPDATE Processing ->|
|                        |                | transcode...    |
|                        |                |                |
|                        |— UPDATE Ready —>| (transcode done) |
|                        |                |                |
# Note: current code sets Ready only after full transcode completes.
# Ready/Completed split (progressive) is production design — not yet
coded.
```

4.2 Stream Flow

Step 1 — Poll for status (Browser → Backend → PostgreSQL):

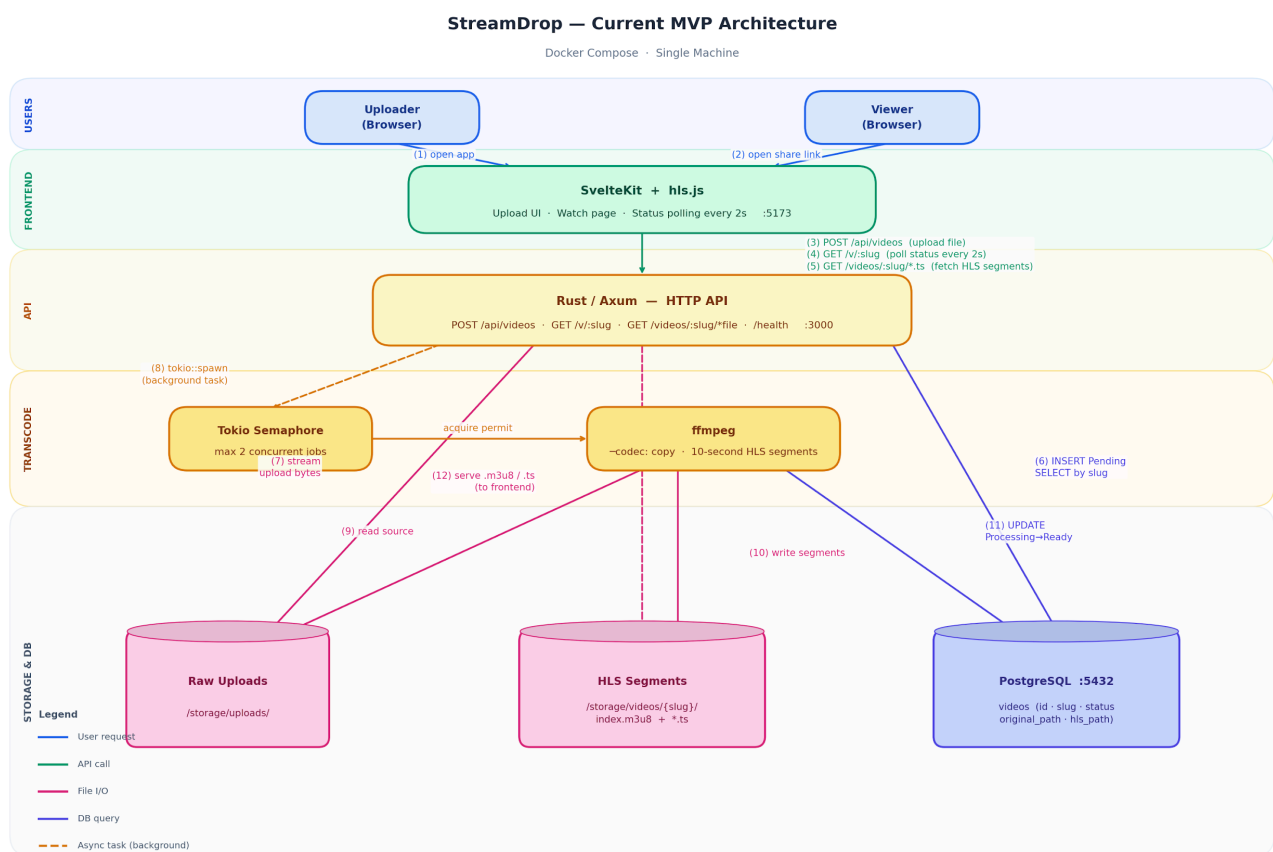
```
Browser (viewer)        Backend                PostgreSQL
|                        |                |
|— GET /v/:slug —>|— SELECT by slug —>|
|<— { status,         |<— { video row } —>|
|   manifest_url }    |                |
|                        |                |
| (repeat every 2s)    |                |
| (stop when Ready)    |                |
```

Step 2 — Fetch HLS segments (Browser → Backend only, no DB):

```
Browser (viewer)           Backend (reads /storage/videos/)
|                           |
| — GET /videos/:slug/    |
|   index.m3u8 ———>      |
|<— (m3u8 bytes) ————|
|                           |
| — GET /videos/:slug/    |
|   seg001.ts ———>      |
|<— (.ts bytes) ————|
|   ... (hls.js fetches segments as playback advances)
```

5. High Level Design — Current MVP

The system runs as a three-container Docker Compose stack on a single machine. Each component is isolated and the storage layer is abstracted behind a trait, making the production upgrade path incremental rather than a rewrite.



Component	Role
Rust / Axum	HTTP API. Streams uploads to disk, manages transcoding via semaphore (max 2), serves HLS, updates DB.
SvelteKit	Upload UI. Watch page polls status, plays via hls.js once Ready.
PostgreSQL	Single videos table. Source of truth for all video metadata and status.
ffmpeg	Runs in backend container. Transcodes to HLS using stream copy (-codec: copy) for maximum speed.
StorageBackend trait	Async trait abstracting all file I/O. LocalStorage today → S3Storage in production with zero business logic changes.

6. Deep Dives

6.1 Why HLS

- Progressive playback: viewer starts after first 2-second segments, not full download.
- CDN-friendly: each segment is an independent cacheable file — impossible with a single MP4.
- Browser compatible: native in Safari, hls.js in Chrome/Firefox.
- Future adaptive bitrate: multiple quality tracks. Architecture supports this without change.

6.2 StorageBackend Trait — Most Important Structural Decision

Every file operation goes through this async trait. Swapping to S3 in production = implement S3Storage + change one line in main.rs. Zero changes to upload handler, transcode service, or routes.

6.3 VideoRepository Trait — Testability

Recommended improvement: abstract db::videos behind a VideoRepository trait. This is not about making Postgres swappable for a different database — it is about testability. MockRepository in unit tests, PostgresRepository in production. A different DB would require rewriting all query logic anyway, so a trait does not save you there.

6.4 ffmpeg: Stream Copy vs Re-encode

Approach	Tradeoff
Current: -codec: copy (stream copy)	Extremely fast (real-time). Risk: exotic source codec not browser-compatible.
Production: -c:v libx264 -c:a aac	Slower. Guaranteed H.264+AAC output. Every browser can play it.

6.5 Known Gaps: What Is Documented vs What Is Coded

Transparency note

These features are described in the production design but are NOT yet implemented. They represent the intended production architecture.

Feature	Status
Ready vs Completed split	NOT coded. Current code sets Ready after full transcode only. Progressive transcoding is future work.
S3 multipart presigned upload	NOT coded. Current upload is single POST to backend.
Dedicated transcoding worker	NOT coded. Transcoding runs as Tokio task in API process.
SQS job queue	NOT coded. Tokio semaphore handles concurrency.
Redis status cache	NOT coded. Every poll hits Postgres directly.
RDS Proxy	Not applicable to MVP.
VideoRepository trait	NOT coded. Direct db:: module calls used.
share_url env var	NOT coded. Hardcoded to localhost:5173.

PART 2

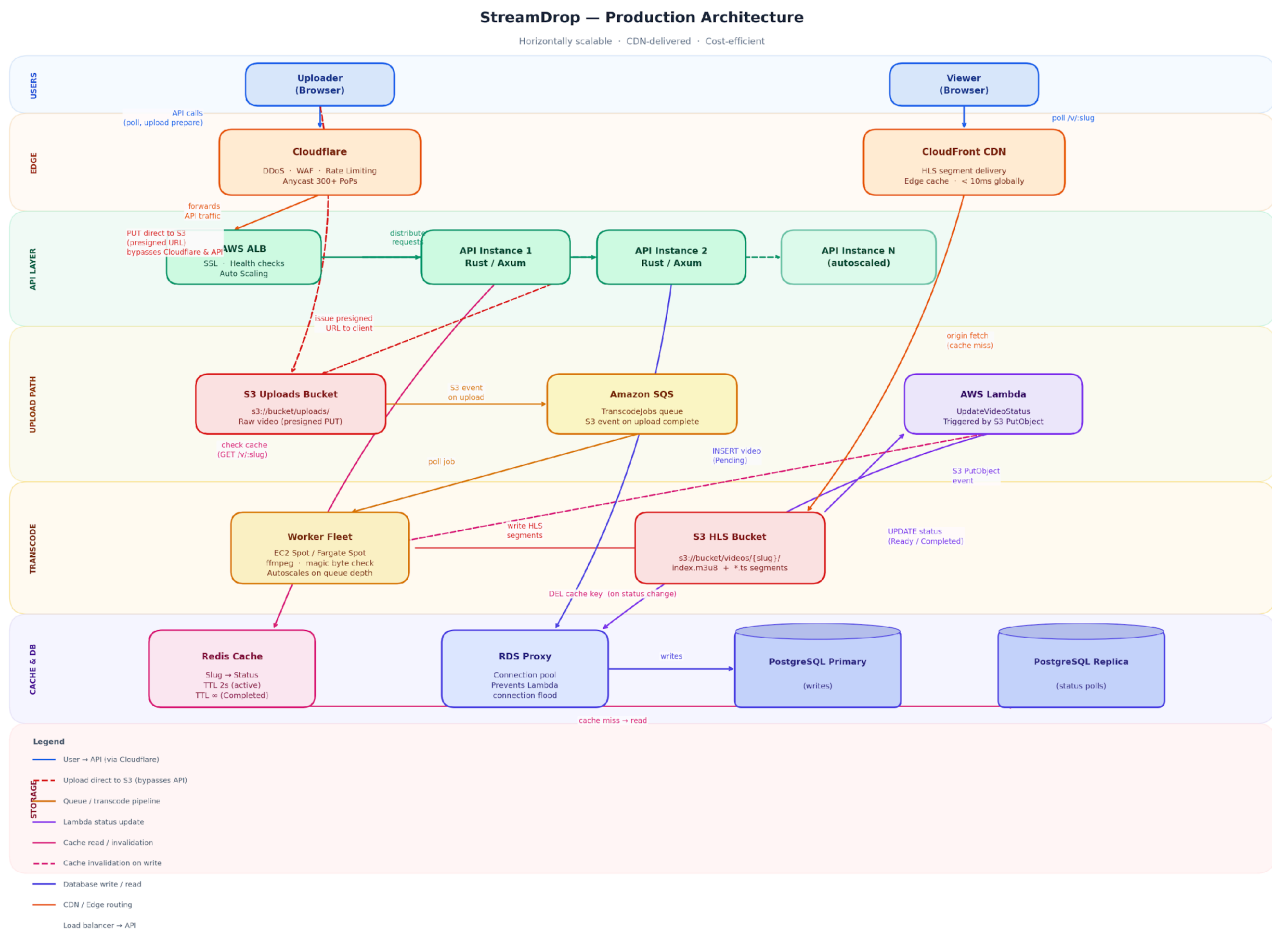
Scaling to Production

What breaks, when, and how to fix it — layer by layer

The MVP runs fine for tens of users. As traffic grows, layers break in a predictable order. This section tells that story — what breaks first, what you add, and when you actually need it. Don't add complexity before you hit the bottleneck.

7. High Level Design — Full Production

The production architecture separates the API, upload path, transcoding, caching, and video delivery into independent scalable layers. Each can be scaled or replaced without touching the others.



Key insight: the API only handles metadata

In production the API never touches video bytes. Uploads go direct to S3 via presigned URL. Streams go direct from CloudFront. Status reads hit Redis first. This is why the API stays small and cheap at any scale.

8. Production Request Flow

API calls:

```
Browser → Cloudflare → AWS ALB → API Instance
    → Redis cache hit: return status immediately
    → Redis cache miss: read replica → cache result → return
```

Upload (video bytes never touch the API):

```
Browser → API (get presigned URLs) → Browser PUT direct to S3
S3 event → SQS → Worker → transcode → write HLS to S3
S3 PutObject event → Lambda → RDS Proxy → Postgres UPDATE status
Lambda also → DEL Redis cache key (invalidate on write)
```

Stream:

```
hls.js → CloudFront CDN → S3 (no API, no Cloudflare)
```

9. Cloudflare

What: Global reverse proxy. Sits in front of everything. Your backend IP is never exposed.

Why

- DDoS protection — absorbs L3/L4 and L7 floods before they reach AWS. You pay Cloudflare, not AWS, for blocked traffic.
- WAF — blocks OWASP Top 10 patterns at the edge before your code runs.
- Anycast routing — 300+ PoPs globally. Tokyo user connects to Tokyo, not your AWS region. Saves 50-200ms on API calls.
- Origin hiding — attackers cannot find your ALB IP and bypass Cloudflare directly.
- Rate limiting — block IPs exceeding upload thresholds before they hit your infrastructure.

What Cloudflare does NOT handle here

- Video uploads bypass Cloudflare entirely — browser uploads direct to S3 via presigned URL to avoid the 100MB proxy limit on Free/Pro plans.
- Video streaming bypasses Cloudflare — hls.js fetches from CloudFront directly. No need for two CDN hops.

When you need it

Add from day one

Free tier covers DDoS, WAF, and rate limiting. 30-minute setup. The cost of not having it during a DDoS is much higher than setup time.

Tradeoffs

Consideration	Detail
Upload limit	100MB proxy limit on Free/Pro. Solved by presigned S3 URLs that bypass Cloudflare.
DNS dependency	Your DNS must point to Cloudflare. Low migration risk — Cloudflare is highly reliable.
Status endpoint caching	Set Cache-Control: no-store on /v:/slug — status changes every few seconds during transcoding.

10. Load Balancer (AWS ALB)

What: Distributes HTTP requests across API instances. Handles SSL, health checks, zero-downtime deploys.

Why ALB and not API Gateway

	AWS ALB
Best for	Long-lived containers
Latency	< 1ms
Cost (10M req/mo)	~\$20
Auto Scaling	Native ASG
Decision	Yes

When you need it

Threshold

A t4g.small (2 vCPU, 2GB) handles ~500-1000 concurrent connections for a lightweight Rust API. Add ALB + Auto Scaling Group when CPU is consistently > 70%, or when you need zero-downtime deploys.

Scaling signals

- Scale out: sustained CPU > 70%, or p99 latency > 500ms, or error rate rising.
- Scale in: CPU < 20% for 10+ minutes. Auto Scaling Group handles this automatically.

11. Upload Design (S3 Multipart + Presigned URLs)

The problem: MVP streams bytes through the API to local disk. Two issues at scale: (1) multiple API instances cannot share a filesystem, (2) the API becomes a bandwidth bottleneck for GBs of video.

Solution: presigned S3 multipart upload

```
POST /api/videos/prepare → validate → CreateMultipartUpload → return parts[]
Browser PUT part 1 direct to S3 (parallel, bypasses API and Cloudflare)
Browser PUT part 2 direct to S3
Browser PUT part N direct to S3 (retry only failed parts individually)
POST /api/videos/complete → CompleteMultipartUpload → SQS message
```

Why multipart

Feature	Why it matters
Per-part retry	Only the failed 10MB chunk retries, not the entire 1GB file.
Parallel uploads	Browser uploads multiple chunks simultaneously — much faster on good connections.
Resume after disconnect	Upload ID persists on S3. Browser resumes from last good part.
Required > 5GB	S3 single PUT is capped at 5GB. Multipart handles 5TB.

Size enforcement — two layers

- Layer 1: API validates declared `size_bytes` before issuing any presigned URLs. Returns 413 immediately.
- Layer 2: content-length-range condition cryptographically signed into each presigned URL. S3 rejects actual bytes that exceed the range. Client cannot override.

File type validation — three layers

Layer	What
API MIME check	<code>mime_type</code> starts with <code>video/</code>
Worker magic bytes	First 12 bytes match known video signatures
ffmpeg probe	Valid container + video stream, not truncated

When you need this

As soon as you add a second API instance. The `StorageBackend` trait means the code is ready — it is a new `S3Storage` implementation plus two new API endpoints.

12. Storage (Local → S3)

The problem: Local filesystem cannot be shared across API instances or workers.

- Shared: every API instance and every worker reads/writes the same S3 bucket.
- Durable: 11 nines durability. Not achievable on a single container disk.
- CDN-compatible: CloudFront integrates natively. HLS segments served from edge.
- Cost-tiered: lifecycle rules move old videos to cheaper storage automatically.

Age	Storage tier
0-30 days	S3 Standard
30-90 days	S3 Standard-IA (40% cheaper)
90+ days	S3 Glacier (90% cheaper)

When you need it

As soon as you run two API instances. The StorageBackend trait makes this a one-implementation, one-config-line change.

13. Transcoding at Scale

The problem: ffmpeg running inside the API process starves the API of CPU. A crash loses in-progress jobs.

Solution: SQS + Worker Fleet + Lambda status update

```
S3 upload complete → S3 Event → SQS (TranscodeJobs)

Worker (EC2 Spot / Fargate Spot):
1. Poll SQS
2. Download file from S3
3. Magic byte check (reject before ffmpeg)
4. ffmpeg transcode to HLS
5. Write each segment to S3 as produced
   → S3 PutObject → Lambda → RDS Proxy → UPDATE status
   First segment → status = Ready (viewer can play NOW)
   Last segment → status = Completed (full video done)
6. Delete SQS message

On crash: SQS visibility timeout expires → message reappears → retry
```

Progressive transcoding — the biggest time-to-stream win

Approach	Time-to-stream
Current MVP	Upload time + full transcode. ~5-15 min for 1GB.
Production (progressive)	~15 seconds from upload start, any file size.

Pipe upload stream directly to ffmpeg stdin. ffmpeg writes segments as it receives data. Lambda fires on first segment and sets Ready. Viewer starts watching while rest is still uploading.

Worker sizing

Uploads/day	Workers
< 100	1-2
100-500	2-5
500+	Autoscale on SQS depth

14. Database

Current state: Single PostgreSQL instance. One table, two indexes. Correct for low traffic.

What breaks first — status polling

Every viewer polls `/v/:slug` every 2 seconds until Ready. At 1000 concurrent viewers across 100 videos being transcoded, that is ~500 reads/second. A `db.t4g.micro` handles ~1000-2000 simple reads/second — so this is not an immediate bottleneck but it is the first thing that shows pressure.

Fix 1: Redis cache (add before read replica)

Cache the GET `/v/:slug` response in Redis with a short TTL. Most status polls hit cache instead of Postgres. This is cheaper and faster than a read replica for this specific access pattern.

```
Poll request arrives for slug xK92mPqR:
```

1. Check Redis: GET `cache:slug:xK92mPqR`
 - Cache hit? Return immediately. No DB query.
 - Cache miss? Query Postgres, store in Redis, return.

```
Cache TTL strategy:
```

```
status = Pending / Processing → TTL 2 seconds (changes frequently)
status = Ready / Failed      → TTL 5 seconds (changes once more or never)
status = Completed           → TTL forever (never changes again)
```

```
Cache invalidation on write:
```

```
Lambda calls UPDATE status on Postgres
Lambda also calls DEL cache:slug:{slug} in Redis
→ Next poll hits Postgres once, refreshes cache
→ Prevents viewer seeing stale status for up to TTL seconds
```

```
# With 1000 viewers polling 100 videos: ~500 req/s
# Redis handles 100,000+ req/s on a single node.
# DB read load drops by ~95%.
```

Why invalidate on write and not just rely on TTL expiry

TTL alone means a viewer could see stale status for up to 2 seconds after the real transition happens. For Pending/Processing this is fine. But for the Ready transition — when the viewer is actively waiting to start watching — you want them to see it within one poll cycle. Invalidating the cache key on each status update ensures the next poll always gets fresh data.

Fix 2: Read Replica (add when Redis is not enough)

A read replica replicates all writes from the primary with ~10ms lag. Route all SELECT queries to the replica. All INSERTs and UPDATEs remain on the primary.

Operation	Route to
INSERT video row	Primary (once per upload)
UPDATE status (via Lambda)	Primary (2-3 times per video)
SELECT by slug (polling)	Redis → Read Replica on cache miss
SELECT by id	Redis → Read Replica on cache miss

Fix 3: RDS Proxy (essential when Lambda writes to DB)

Each Lambda invocation is ephemeral — it can open a new Postgres connection on every call. At 100 concurrent Lambda invocations (100 segments being written simultaneously across multiple videos), that is 100 new connections. RDS db.t4g.micro supports ~80 max connections total. Without RDS Proxy this crashes the database.

```

Without RDS Proxy:
  Lambda invocation 1 → new Postgres connection
  Lambda invocation 2 → new Postgres connection
  Lambda invocation N → connection limit hit → DB refuses connections →
  crash

With RDS Proxy:
  Lambda invocation 1 → RDS Proxy (pooled connection 1 of 20)
  Lambda invocation 2 → RDS Proxy (pooled connection 2 of 20)
  Lambda invocation N → RDS Proxy (queues if all 20 busy, does not crash
  DB)
  Lambda completes → connection returned to pool

# RDS Proxy sits between Lambda and RDS.
# Lambda talks to Proxy endpoint, not directly to RDS.
# Proxy maintains a fixed connection pool to RDS.
# Cost: ~$0.015/vCPU-hour of your RDS instance. ~$5-10/month.

```

When you need RDS Proxy

As soon as Lambda is writing to the database. Without it, a burst of S3 events (many segments written at once) creates a burst of Lambda invocations, which creates a burst of DB connections, which hits the connection limit and crashes Postgres. RDS Proxy is not optional here — it is essential.

Scaling signals

- Add Redis when: status polling is > 20% of DB CPU, or you have > 100 concurrent viewers.
- Add read replica when: Redis miss rate is high (cache TTLs too short relative to poll rate), or DB CPU > 60%.
- Add RDS Proxy: always, as soon as Lambda writes to Postgres.
- Add PgBouncer: when instance count x pool size approaches DB max connections for non-Lambda clients.

15. Video Delivery (CloudFront CDN)

The problem: MVP API serves .ts segment files directly. 100 viewers x 500 segments per video = 50,000 requests/hour to the API. The API is not a file server.

CloudFront caches HLS segments at the edge after the first request. Every subsequent viewer gets the segment from the nearest PoP — typically < 10ms — with zero backend involvement.

```
First viewer (cache miss):  
  hls.js → CloudFront Singapore → S3 origin → cached → served (~100-200ms)  
  
All subsequent viewers (cache hit):  
  hls.js → CloudFront Singapore → served from edge (< 10ms globally)  
  
# This is what makes playback consistent regardless of file size.  
# A 10MB and a 1GB video stream at the same speed once edge-cached.  
# Cache hit rate: 90-99% for any video with more than a few viewers.
```

Delivery method	Cost per GB
API serves files	EC2 bandwidth + CPU
S3 direct	\$0.09/GB
CloudFront → S3	\$0.008-0.02/GB + \$0.0004/10k requests

When you need it

As soon as you move to S3. Without CloudFront every segment request hits S3 at \$0.09/GB. With CloudFront, popular videos cost almost nothing to serve after the first viewer.

16. Rate Limiting

Without user accounts, rate limiting is IP-based across four layers.

Layer 1 — Cloudflare

- POST /api/videos/prepare: 10 req/IP/hour. Returns 429 before reaching AWS.
- Adjust threshold based on real usage patterns. 10/hour = 240 uploads/day per IP — generous for legitimate use, blocks automated flooding.

Layer 2 — S3 presigned URL scoping

- Each URL: one S3 key, PUT only, expires in 60 minutes.
- content-length-range condition: 10MB max per part. Signed by AWS — client cannot override.
- Leaked URL is useless after expiry and cannot upload to a different path.

Layer 3 — SQS as natural throttle

- Jobs queue rather than spawning workers immediately. Burst uploads queue, not crash.
- Hard worker fleet maximum (e.g. 10 workers) caps transcoding cost even under attack.

Layer 4 — Application checks

- size_bytes validated before presigned URLs issued. 413 before any S3 cost.
- S3 lifecycle rule: abort incomplete multipart uploads after 24h. Prevents orphan part accumulation.

17. Upload & Security Deep Dives

This section covers production-quality design decisions for upload integrity, security, compression, and efficient sync. These are not in the MVP but are the right answers to 'how would you do this properly?' in a design review.

18.1 File Fingerprinting — Deduplication and Resumability

A file fingerprint is a SHA-256 hash of the file's full content computed on the client before upload begins. It is stored as a field on the video metadata record alongside the upload_id. Two separate uses:

- Deduplication: before the client calls /prepare, it sends the fingerprint. If the API finds an existing record with the same fingerprint and status=Completed, it skips the entire upload — no presigned URLs, no S3 writes, no SQS message. A new metadata record is created pointing to the already-existing S3 object. Two users uploading the same 1GB video pay the storage cost once.
- Resumability: if the client finds an existing record with the same fingerprint and status=Uploading, the API returns the existing upload_id and calls S3 ListParts to return which parts S3 already received. The client skips those parts and only uploads the remainder. No restarting from zero after a dropped connection.

Fingerprint vs video_id

The fingerprint identifies content. Two users uploading identical video produce the same fingerprint but get separate video_id, slug, and access records. The fingerprint is a dedup lookup key, not the primary identifier. Never expose it in the share URL.

18.2 ETag Verification — Trust But Verify

S3 returns an ETag for each successfully uploaded part. The client collects these and sends them in the /complete request. A naive implementation passes them directly to CompleteMultipartUpload — but this trusts the client entirely. A malicious actor could submit fake ETags to mark a video as uploaded without actually sending any bytes, producing a corrupt record in the database.

The fix: before calling CompleteMultipartUpload, the API calls S3 ListParts with the upload_id. S3 returns the real ETags for every part it actually received. The API cross-references these against the client-submitted ETags. If they match, CompleteMultipartUpload proceeds. If they differ or parts are missing, the request is rejected with 409 and the upload_id remains open for retry.

```
Client → POST /api/videos/complete { upload_id, parts: [{part_number, etag}]
}
API    → S3 ListParts(upload_id)
S3     → [{ part_number, etag, size }, ...]
API    → compare client ETags against S3 ETags
        match: S3 CompleteMultipartUpload → SQS → transcode
        mismatch: 409 Conflict, upload_id left open, client retries missing
parts

# Cost: one extra S3 API call per completed upload.
# Result: guaranteed consistency between what client claims and what S3
holds.
```

18.3 Presigned URLs Are Bearer Tokens

A presigned URL embeds the access grant in the URL itself. Anyone who holds the URL can use it — there is no separate authentication step. This is a deliberate design choice for performance (no auth round-trip), but it has an important implication: if an authorized user leaks the URL, the file is accessible to anyone until the URL expires.

The TTL is the primary defence. For StreamDrop:

- Upload URLs: 60-minute TTL — large files need time to upload. Acceptable because upload URLs are scoped to one S3 key + PUT only. A leaked upload URL can only overwrite that specific file path.
- Download/stream URLs: 5-minute TTL — tight enough to limit exposure, long enough for a player to buffer. The CDN revalidates after expiry.

For scenarios requiring tighter control, two additional restrictions can be embedded into the presigned policy at generation time:

- IP binding: the policy restricts the PUT or GET to the IP address of the requesting client. S3 rejects requests from any other IP. Suitable for desktop clients; avoid for mobile (IP changes frequently on cellular networks).
- Single-use tokens: the API generates a one-time nonce alongside the URL. On first use the token is invalidated. Any replay — even within the TTL — fails. Adds one Redis lookup per request.

CloudFront signed URLs follow the same model

CloudFront signed URLs for streaming also embed the grant in the URL. Same bearer token caveat applies. The 5-minute TTL is why hls.js needs to request a fresh manifest URL before each playback session — not just once at share-link creation time.

18.4 Client-Side Compression

Compressing video files before upload saves bandwidth and reduces upload time. Because uploads go directly from the client to S3, compression must happen entirely on the client — the API is never in the data path for file bytes.

Two rules that always apply:

- Compress before encrypt: encryption introduces near-random byte patterns that are incompressible. Compressing after encryption achieves essentially 0% reduction. Always compress first, then encrypt, or the CPU time is wasted.
- Skip already-compressed formats: video, image, and audio files are already compressed internally (H.264, HEVC, AAC, JPEG, PNG). Re-compressing them typically increases file size by 1-2%. Skip compression for all video/*, image/*, and audio/* MIME types.

Algorithm	Compression
Gzip	Good
Brotli	Better
Zstandard (zstd)	Good

For StreamDrop the practical decision is a single MIME prefix check before the chunk loop starts. If MIME starts with video/, image/, or audio/ — skip compression. For everything else (subtitle files, .vtt captions, .srt, metadata) — apply Zstandard level 3 (default). The compression/decompression is transparent to the API and workers.

18.5 Content-Defined Chunking and Delta Sync

The standard 10MB fixed-part multipart upload works well for initial uploads. It becomes a problem the moment files can be updated. A single byte inserted near the start of a 1GB file

shifts every subsequent 10MB chunk boundary — producing N new fingerprints for N identical-content chunks. The client re-uploads the entire file to sync a trivial edit.

Content-Defined Chunking (CDC)

CDC determines chunk boundaries from the file's content rather than fixed byte offsets. A rolling hash (Rabin fingerprinting) scans the file byte-by-byte. When the rolling hash value matches a specific bitmask pattern, that byte position becomes a chunk boundary. Because boundaries are content-derived, editing one region of a file only changes the one or two chunks surrounding that region. All other chunks retain their exact fingerprint.

```
Fixed-size chunking:
  Insert 1 byte at position 0 → all N chunks shift
  All N fingerprints change → entire file re-uploaded

Content-Defined Chunking (Rabin):
  Insert 1 byte at position 0 → 1-2 boundary chunks change
  N-2 chunks unchanged → only ~10-20MB re-uploaded

# This is how Dropbox achieves sub-second delta sync on large files.
# Target chunk size: 4-8MB average, with min 1MB / max 16MB bounds.
```

CDC produces variable-size chunks. The average target size (e.g. 8MB) is controlled by the bitmask width — a 23-bit mask produces an average boundary every $2^{23} = 8\text{MB}$ bytes. The min and max bounds prevent degenerate tiny or enormous chunks that would hurt parallelism or S3 limits.

When this matters for StreamDrop

StreamDrop currently only supports initial upload — no file updates. CDC is irrelevant today. It becomes the right design the moment any 'replace video' or 'update file' feature is added. Noting it in a design review shows awareness of the problem before it is hit.

Encryption at rest

S3 server-side encryption (SSE-S3) encrypts every object on write using AES-256. The key is managed by AWS, rotated regularly, stored separately from the data. No code change required — enable at the bucket level and every object (existing and future) is encrypted at rest.

SSE-KMS uses AWS Key Management Service instead. You own the key rotation policy and get a full audit trail of every decryption event via CloudTrail. Costs ~\$0.03 per 10,000 API calls. Use SSE-KMS when compliance requires audit-level access logging; SSE-S3 is sufficient for general use.

In transit vs at rest

TLS 1.2+ (enforced by S3's HTTPS endpoint) covers data in flight between client and S3 via presigned URLs. SSE-S3 covers data on S3 disk. Both should be enabled. Neither requires any change to backend code.

18. When to Add What — Summary

Stage	Signal	Add
Day 1	Any traffic	Cloudflare + single instance + MVP as-is
Early growth	~100 uploads/day	S3 storage, presigned upload, CloudFront CDN
Moderate	~500 uploads/day or transcode lag visible	SQS + worker fleet + Lambda + RDS Proxy
DB pressure	> 100 concurrent viewers polling	Redis status cache (before read replica)
API scaling	CPU consistently > 70%	ALB + Auto Scaling Group + 2nd API instance
DB read scale	Redis miss rate high or DB CPU > 60%	Postgres read replica
Cost optimise	Storage costs growing	S3 lifecycle tiers, spot workers

The guiding principle

Each layer solves a specific bottleneck. The StorageBackend trait and Tokio semaphore in the current code are already designed for these upgrades. The path from MVP to full production is incremental — each step replaces one component, not the whole system.